Assignment-17

Name: Akshaya Nemalipuri

Batch: 37

Roll No: 2303A2441

Lab: 17.2

Lab 17 – AI for Data Processing: Data Cleaning and Preprocessing

Task 1: Student Academic Data Cleaning

• Task:

Use AI assistance to generate a Python script that cleans and

prepares a student academic dataset for analysis.

Instructions:

• Handle missing values in marks, attendance, and department.

• Remove duplicate student records based on student ID.

• Standardize text fields such as department names.

• Encode categorical variables like gender and department.

Sample Code:

```
import pandas as pd

data = pd.read_csv("students.csv")

print(data.head())
```

Expected Output:

• A cleaned and encoded Pandas DataFrame suitable for academic

performance analysis.

Prompt :

Generate a Python script using pandas to create and clean a student dataset. Include handling missing values, duplicates, standardization, and encoding.

Code :

```python
#Generate a Python script using pandas and sklearn. Step 1: Create a sample CSV file named "students.csv" with at least 20 rows containing
import pandas as pd
from sklearn.preprocessing import LabelEncoder

# Step 1: Create sample CSV data
data = [
    {"student_id": 1, "name": "Alice",    "gender": "Female", "marks": 85, "attendance": 92, "department": "CSE"},
    {"student_id": 2, "name": "Bob",      "gender": "Male",   "marks": 78, "attendance": 88, "department": "cse "},
    {"student_id": 3, "name": "Carol",    "gender": "Female", "marks": 90, "attendance": None, "department": " ECE"},
    {"student_id": 4, "name": "David",    "gender": "Male",   "marks": None, "attendance": 75, "department": "ME"},
    {"student_id": 5, "name": "Eva",      "gender": None,     "marks": 82, "attendance": 80, "department": "cse"},
    {"student_id": 6, "name": "Frank",    "gender": "Male",   "marks": 81, "attendance": 95, "department": "ECE "},
    {"student_id": 7, "name": "Grace",    "gender": "Female", "marks": 69, "attendance": 78, "department": "ME"},
    {"student_id": 8, "name": "Henry",    "gender": "Male",   "marks": 74, "attendance": None, "department": None},
    {"student_id": 9, "name": "Ivy",      "gender": "Female", "marks": 88, "attendance": 90, "department": "CSE"},
    {"student_id":10, "name": "James",    "gender": "Male",   "marks": 56, "attendance": None, "department": "ece"},
    {"student_id":11, "name": "Kathy",    "gender": None,     "marks": 77, "attendance": 82, "department": "CSE"},
    {"student_id":12, "name": "Liam",     "gender": "Male",   "marks": 92, "attendance": 98, "department": " ece"},
    {"student_id":13, "name": "Mona",     "gender": "Female", "marks": None, "attendance": 80, "department": "ME"},
    {"student_id":14, "name": "Nate",     "gender": "Male",   "marks": 83, "attendance": 86, "department": "cse"},
    {"student_id":15, "name": "Olga",     "gender": "Female", "marks": 89, "attendance": 90, "department": " ECE"},
    {"student_id":16, "name": "Paul",     "gender": "Male",   "marks": 88, "attendance": 85, "department": None},
    {"student_id":16, "name": "Paul Dup", "gender": "Male",   "marks": 88, "attendance": 85, "department": None},  # duplicate student_id
    {"student_id":17, "name": "Quinn",    "gender": "Female", "marks": 93, "attendance": 93, "department": "CSE"},
    {"student_id":18, "name": "Rita",     "gender": "Female", "marks": None, "attendance": None, "department": "ece"},
    {"student_id":19, "name": "Steve",    "gender": "Male",   "marks": 79, "attendance": 81, "department": "ME"},
    {"student_id":20, "name": "Tina",     "gender": "Female", "marks": 86, "attendance": 87, "department": "cse "}
]

df = pd.DataFrame(data)
```

```
PROBLEMS 1   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                                   Python

9          10  James     Male  56.000000        88.0              1    False     True    False
```

```python
df = pd.read_csv("students.csv")
print("Original data preview:")
print(df.head(10))
print()

# Fill missing marks with mean
marks_mean = df["marks"].mean()
df["marks"] = df["marks"].fillna(marks_mean)

# Fill missing attendance with median
attendance_median = df["attendance"].median()
df["attendance"] = df["attendance"].fillna(attendance_median)

# Fill missing department with mode
dept_mode = df["department"].mode().iloc[0]
df["department"] = df["department"].fillna(dept_mode)

# Remove duplicates based on student_id (keep first)
df = df.drop_duplicates(subset=["student_id"], keep="first")

# Standardize department (lowercase + strip)
df["department"] = df["department"].astype(str).str.strip().str.lower()

# Encode gender with label encoding (fill any missing first)
df["gender"] = df["gender"].fillna("Unknown")
le = LabelEncoder()
df["gender_encoded"] = le.fit_transform(df["gender"])
```

```python
df = pd.get_dummies(df, columns=["department"], prefix="dept", dummy_na=False)

# Step 3: Display cleaned dataset and info
print("Cleaned and encoded dataset preview:")
print(df.head(10))
print()
print("DataFrame info:")
print(df.info())
print()
print("Value counts for gender encoding:")
print(df["gender"].value_counts())
print("Label encoding mapping:", dict(zip(le.classes_, le.transform(le.classes_))))
```

```
0          1  Alice   Female  85.000000        92.0              0      True     False    False
1          2    Bob     Male  78.000000        88.0              1      True     False    False
2          3  Carol   Female  90.000000        88.0              0     False      True    False
3          4  David     Male  82.666667        75.0              1     False     False     True
4          5    Eva  Unknown  82.000000        80.0              2      True     False    False
5          6  Frank     Male  91.000000        95.0              1     False      True    False
6          7  Grace   Female  69.000000        70.0              0     False     False     True
7          8  Henry     Male  74.000000        88.0              1      True     False    False
8          9    Ivy   Female  88.000000        90.0              0      True     False    False
9         10  James     Male  56.000000        88.0              1     False      True    False

DataFrame info:
<class 'pandas.core.frame.DataFrame'>
Index: 20 entries, 0 to 20
Data columns (total 9 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   student_id      20 non-null     int64
 1   name            20 non-null     object
 2   gender          20 non-null     object
 3   marks           20 non-null     float64
 4   attendance      20 non-null     float64
 5   gender_encoded  20 non-null     int64
 6   dept_cse        20 non-null     bool
 7   dept_ece        20 non-null     bool
 8   dept_me         20 non-null     bool
dtypes: bool(3), float64(2), int64(2), object(2)
memory usage: 1.2+ KB
None

Value counts for gender encoding:
gender
Female     9
Male       9
Unknown    2
Name: count, dtype: int64
 8   dept_me         20 non-null     bool
dtypes: bool(3), float64(2), int64(2), object(2)
```

Explanation

In this task, a student dataset is created and cleaned using preprocessing techniques. Missing values in marks, attendance, and department are handled using statistical methods such as mean, median, and mode. Duplicate records are removed based on student ID to ensure data integrity. Text fields such as department names are standardized to maintain consistency. Finally, categorical variables like gender and department are encoded into numerical form for analysis.

Task 2: Financial Transaction Data Preprocessing :

Task:

Preprocess financial transaction data using AI-generated Python

code.

Instructions:

• Convert transaction date columns to datetime format.

• Create derived features such as transaction month and year.

• Normalize the transaction amount column.

• Identify and handle extreme transaction values.

Sample Code:

```
import pandas as pd

from sklearn.preprocessing import MinMaxScaler

transactions = pd.read_csv("transactions.csv")

print(transactions.info())
```

Expected Output:

• A transformed dataset with normalized values and newly

generated time-based features.

Prompt :

Generate Python code to preprocess transaction data including datetime conversion, feature extraction, normalization, and outlier handling.

Code :

```python
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler

# Step 1: Create sample CSV data
np.random.seed(42)
n = 35
base_date = pd.to_datetime("2023-01-01")
data = []
for i in range(1, n + 1):
    tx_id = 1000 + i
    tx_date = (base_date + pd.to_timedelta(np.random.randint(0, 365), unit="d"
    amount = np.random.normal(200, 80)  # normal cluster
    if np.random.rand() < 0.1:
        amount = np.random.choice([1500, 3000, 5000, -200])  # extreme outlier
    customer_id = np.random.randint(1, 15)
    data.append({
        "transaction_id": tx_id,
        "transaction_date": tx_date,
        "transaction_amount": round(amount, 2),
        "customer_id": customer_id,
    })

df = pd.DataFrame(data)
df.to_csv("transactions.csv", index=False)
print("Created transactions.csv\n")

# Step 2: Preprocess dataset
df = pd.read_csv("transactions.csv")
print("=== Before Preprocessing ===")
print(df.describe(include="all"))
print(df.head(5))
print()
```

```python
# Convert transaction_date to datetime
df["transaction_date"] = pd.to_datetime(df["transaction_date"], format="%Y-%m-

# Create new features
df["transaction_month"] = df["transaction_date"].dt.month
df["transaction_year"] = df["transaction_date"].dt.year

# Normalize transaction_amount using MinMaxScaler
scaler = MinMaxScaler()
df["transaction_amount_norm"] = scaler.fit_transform(df[["transaction_amount"]

# Detect outliers using IQR and cap extreme values
q1 = df["transaction_amount"].quantile(0.25)
q3 = df["transaction_amount"].quantile(0.75)
iqr = q3 - q1
lower_bound = q1 - 1.5 * iqr
upper_bound = q3 + 1.5 * iqr

# Save original values for comparison
df["transaction_amount_capped"] = df["transaction_amount"].copy()
df.loc[df["transaction_amount_capped"] < lower_bound, "transaction_amount_capp
df.loc[df["transaction_amount_capped"] > upper_bound, "transaction_amount_capp

# Re-normalize after capping
df["transaction_amount_norm_capped"] = scaler.fit_transform(df[["transaction_a

# Step 3: Summary and final output
print("=== Calculated bounds ===")
print(f"IQR: {iqr:.2f}, lower_bound: {lower_bound:.2f}, upper_bound: {upper_bo
print()

print("=== After Preprocessing ===")
print(df.describe(include="all"))
print(df.head(8))
```

```
=== Before Preprocessing ===
        transaction_id transaction_date  transaction_amount  customer_id
count        35.000000               35           35.000000    35.000000
unique             NaN               30                 NaN          NaN
top                NaN       2023-04-13                 NaN          NaN
freq               NaN                2                 NaN          NaN
mean       1018.000000              NaN          253.454000     8.028571
std          10.246951              NaN          328.701115     4.422441
min        1001.000000              NaN         -200.000000     1.000000
25%        1009.500000              NaN          122.390000     3.500000
1            1002       2023-04-13              241.23            7
2            1003       2023-11-27               39.12            3
3            1004       2023-05-30             -200.00           12
4            1005       2023-10-21             1500.00            1

=== Calculated bounds ===
IQR: 132.95, lower_bound: -77.03, upper_bound: 454.75

=== After Preprocessing ===
       transaction_id  ... transaction_amount_norm_capped
count       35.000000  ...                      35.000000
mean      1018.000000  ...                       0.515752
min       1001.000000  ...                       0.000000
6            1007       2023-07-07  ...                   143.7300                        0.4151
29
7            1008       2023-12-30  ...                    28.8300                        0.1990
63

[8 rows x 9 columns]

=== Outlier caps count ===
Below lower: 1
Above upper: 2
   transaction_id transaction_date  ...  transaction_year  customer_id
0            1001       2023-04-13  ...              2023            5
1            1002       2023-04-13  ...              2023            7
2            1003       2023-11-27  ...              2023            3
3            1004       2023-05-30  ...              2023           12
```

Explanation :

In this task, financial transaction data is preprocessed to make it suitable for analysis. The transaction date is converted into datetime format to enable time-based analysis. Additional features such as transaction month and year are extracted. The transaction amount is normalized to bring all values into a common scale. Outliers are identified and handled to reduce their impact on analysis.

Task 3: Environmental Sensor Data Preparation

• Task:

Clean and preprocess environmental sensor data using AI-assisted

scripts.

Instructions:

• Handle missing values in temperature, humidity, and

air_quality_index.

• Scale numerical features using standard scaling.

• Encode categorical fields such as sensor_status.

• Split the dataset into training and testing subsets.

Sample Code:

```
import pandas as pd

from sklearn.model_selection import train_test_split

data = pd.read_csv("sensor_data.csv")
```

Expected Output:

• A fully preprocessed dataset ready for predictive modeling tasks.

Prompt :

Generate Python code to clean sensor data by handling missing values, scaling features, encoding categorical data, and splitting dataset.

Code :

```python
# Step 1: Create sample CSV
np.random.seed(42)
n = 28
data = []
for i in range(n):
    temp = np.random.normal(25, 5)
    hum = np.random.normal(50, 10)
    aqi = np.random.normal(80, 15)
    if np.random.rand() < 0.15:
        temp = np.nan
    if np.random.rand() < 0.15:
        hum = np.nan
    if np.random.rand() < 0.15:
        aqi = np.nan
    status = np.random.choice(["active", "inactive", "faulty"], p=[0.6, 0.25,
    data.append({
        "temperature": round(temp if not np.isnan(temp) else np.nan, 2),
        "humidity": round(hum if not np.isnan(hum) else np.nan, 2),
        "air_quality_index": round(aqi if not np.isnan(aqi) else np.nan, 2),
        "sensor_status": status
    })

df = pd.DataFrame(data)
df.to_csv("sensor_data.csv", index=False)
print("Created sensor_data.csv with sample data.\n")

# Step 2: Preprocess
df = pd.read_csv("sensor_data.csv")
print("Original NA counts:")
print(df.isna().sum(), "\n")

# Fill missing numeric with mean
for col in ["temperature", "humidity", "air_quality_index"]:
```

```python
# Fill missing numeric with mean
for col in ["temperature", "humidity", "air_quality_index"]:
    df[col].fillna(df[col].mean(), inplace=True)

# Scale numeric features
scaler = StandardScaler()
num_cols = ["temperature", "humidity", "air_quality_index"]
df[num_cols] = scaler.fit_transform(df[num_cols])

# Encode sensor_status
le = LabelEncoder()
df["sensor_status_encoded"] = le.fit_transform(df["sensor_status"])

# Train/test split 80/20
X = df[num_cols + ["sensor_status_encoded"]]
y = df["sensor_status_encoded"]  # placeholder target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=df["sensor_status_encoded"]
)

# Step 3: Print shapes
print("Train shape:", X_train.shape, y_train.shape)
print("Test shape:", X_test.shape, y_test.shape)
print("\nLabel mapping:", dict(zip(le.classes_, le.transform(le.classes_))))
print("\nSample processed data:")
print(df.head())
```

```
Created sensor_data.csv with sample data.

Original NA counts:
temperature          9
humidity             1
air_quality_index    7
sensor_status        0
dtype: int64

Train shape: (22, 4) (22,)
Test shape: (6, 4) (6,)

Label mapping: {'active': np.int64(0), 'faulty': np.int64(1), 'inactive': np.int64(2)}

Sample processed data:
    temperature  humidity  air_quality_index sensor_status  sensor_status_encoded
0  5.057294e-01  0.065349       9.978517e-16        faulty                      1
1  3.519333e-15  2.151807       7.687487e-01        active                      0
2 -6.829954e-01 -0.333228       2.154342e-01        active                      0
3 -2.478465e+00 -0.870336      -1.526664e+00        active                      0
4  3.519333e-15  0.367928      -1.251411e+00      inactive                      2
PS C:\Users\lenovo>
```

Explanation

This task focuses on preparing environmental sensor data for machine learning. Missing values in numerical columns like temperature, humidity, and air quality index are replaced

using mean values. Numerical features are scaled to improve model performance. Categorical data such as sensor status is encoded into numerical format. Finally, the dataset is split into training and testing sets for model building.

Task 4: Smart City Traffic Data Cleaning :

• Scenario:

A smart city system collects traffic data from multiple sensors,

resulting in noisy and inconsistent records.

Requirements:

• Standardize road and location names.

• Fill missing traffic density values using appropriate statistical

methods.

• Remove duplicate sensor readings.

• Normalize speed and vehicle count metrics.

• Generate a brief summary comparing dataset quality before and

after cleaning

Sample Code:

```
import pandas as pd

traffic = pd.read_csv("traffic_data.csv")

print(traffic.describe())
```

Expected Output:

• A cleaned and normalized traffic dataset with an accompanying

data-quality improvement summary.

Prompt :

Generate Python script to clean traffic dataset including standardization, missing value handling, duplicate removal, normalization, and comparison.

Code :

```python
# Step 1: Create sample CSV
np.random.seed(24)
rows = []
roads = ["Main St", "main st ", "MAIN ST", "Broadway", " broadway", "Highway 1", "highway 1 "]
locs = ["Downtown", "downtown ", "DOWNTOWN", "Uptown", " uptown", "Suburb", "suburb "]

for i in range(1, 36):
    sensor_id = np.random.choice([101, 102, 103, 104, 105], p=[0.2, 0.2, 0.2, 0.2, 0.2])
    r = np.random.choice(roads)
    l = np.random.choice(locs)
    density = np.random.normal(45, 20)
    if np.random.rand() < 0.10:
        density = np.nan
    speed = np.random.uniform(20, 90)
    vehicle_count = int(np.random.poisson(60))
    rows.append({
        "sensor_id": sensor_id,
        "road_name": r,
        "location": l,
        "traffic_density": round(density, 2) if not np.isnan(density) else np.nan,
        "speed": round(speed, 2),
        "vehicle_count": vehicle_count,
    })

df = pd.DataFrame(rows)
df.to_csv("traffic_data.csv", index=False)
print("Created traffic_data.csv with sample data.\n")

# Step 2: Clean and preprocess
df = pd.read_csv("traffic_data.csv")

print("=== Before cleaning ===")
```

```python
median_density = df["traffic_density"].median()
df["traffic_density"] = df["traffic_density"].fillna(median_density)

# Remove duplicates on sensor_id + road_name + location + speed + vehicle_count (sensor reading)
before_dups = len(df)
df = df.drop_duplicates(subset=["sensor_id", "road_name", "location", "speed", "vehicle_count"], keep="first")
after_dups = len(df)

# Normalize speed and vehicle_count with MinMaxScaler
scaler = MinMaxScaler()
df[["speed_norm", "vehicle_count_norm"]] = scaler.fit_transform(df[["speed", "vehicle_count"]])

# Step 3: Compare quality before/after
print("=== Cleaning actions ===")
print(f"Rows before dedupe: {before_dups}, after dedupe: {after_dups}")
print(f"Filled traffic_density NaN with median={median_density:.2f}\n")

print("=== After cleaning ===")
print("Missing values:")
print(df.isna().sum())
print("\nSummary statistics:")
print(df.describe(include="all"))
print()

print("=== Sample cleaned data ===")
print(df.head(10))

print("\n=== Data quality summary ===")
print("Total rows:", len(df))
print("Total cols:", df.shape[1])
print("Columns:", list(df.columns))
```

```
freq           NaN        15        15              NaN        NaN           NaN        NaN              NaN
mean    103.314286       NaN       NaN        42.403143  54.298000      59.942857   0.487422         0.497403
std       1.510189       NaN       NaN        18.942857  19.228563       6.641859   0.278958         0.301903
min     101.000000       NaN       NaN         3.940000  20.700000      49.000000   0.000000         0.000000
25%     102.000000       NaN       NaN        31.395000  42.145000      54.000000   0.311113         0.227273
50%     104.000000       NaN       NaN        43.420000  55.860000      60.000000   0.510083         0.500000
75%     105.000000       NaN       NaN        54.610000  66.795000      65.000000   0.668722         0.727273
max     105.000000       NaN       NaN        77.420000  89.630000      71.000000   1.000000         1.000000

=== Sample cleaned data ===
   sensor_id  road_name  location  traffic_density  speed  vehicle_count  speed_norm  vehicle_count_norm
0        105    main st  downtown            74.76  63.31             55    0.618163            0.272727
1        101    main st    uptown            45.93  57.39             54    0.532279            0.227273
2        103  highway 1  downtown             3.94  55.86             56    0.510083            0.318182
3        105   broadway    uptown            32.88  85.44             49    0.939214            0.000000
4        102    main st  downtown            46.23  43.52             54    0.331060            0.227273
5        102   broadway    suburb            29.62  67.90             59    0.684753            0.454545
6        104    main st  downtown            64.94  23.55             60    0.041346            0.500000
7        104  highway 1    uptown            52.81  58.04             71    0.541709            1.000000
8        101    main st  downtown            35.37  61.78             64    0.595967            0.681818
9        105    main st    uptown             9.96  66.17             52    0.659655            0.136364

=== Data quality summary ===
Total rows: 35
Total cols: 8
Columns: ['sensor_id', 'road_name', 'location', 'traffic_density', 'speed', 'vehicle_count', 'speed_norm', 'vehicle_count_norm']
```

Explanation

In this task, traffic data collected from multiple sensors is cleaned and standardized. Road and location names are converted into a consistent format by removing extra spaces and converting to lowercase. Missing values in traffic density are filled using median values. Duplicate records are removed to avoid redundancy. Numerical features such as speed and vehicle count are normalized to ensure uniform scaling across the dataset.

Task 5: Social Media Text Cleaning and Feature Extraction :

• Task:

Use AI-assisted Python scripting to clean and preprocess social

media text data for sentiment analysis.

Instructions:

• Remove duplicate posts and null text entries.

• Clean text by removing URLs, special characters, and extra

spaces.

• Convert text to lowercase and perform basic tokenization.

• Remove stopwords and perform stemming or lemmatization.

• Generate basic text features such as word count and sentiment

score.

Sample Code:

```
import pandas as pd

import re

posts = pd.read_csv("social_media_posts.csv")

print(posts.head())
```

Expected Output:

A cleaned text dataset with extracted features suitable for sentiment

analysis models.

Prompt :

Generate Python code to clean social media text data without using nltk, including regex cleaning, tokenization, stopword removal, and feature extraction.

Code :

```python
import pandas as pd
import numpy as np
import re
import nltk
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from textblob import TextBlob

# Ensure NLTK resources are available
nltk.download("punkt", quiet=True)
nltk.download("stopwords", quiet=True)
nltk.download("wordnet", quiet=True)
nltk.download("omw-1.4", quiet=True)

# Step 1: create sample CSV
rows = [
    {"post_id": 1, "text": "Love this! #sunnyday https://example.com"},
    {"post_id": 2, "text": "bad service... never again!!"},
    {"post_id": 3, "text": "OMG!!! this is the best thing EVER!! 😍"},
    {"post_id": 4, "text": "Visit https://test.com now :)"},
    {"post_id": 5, "text": None},
    {"post_id": 6, "text": "Just meh..."},
    {"post_id": 7, "text": "JUST got a new camera, So happy."},
    {"post_id": 8, "text": "just got a new camera, so happy."},  # duplicate-like
    {"post_id": 9, "text": "Spam http://spam.org spam spam"},
    {"post_id":10, "text": "I hate this! #fail"},
    {"post_id":11, "text": "i Hate this! #fail"},  # duplicate-like case
    {"post_id":12, "text": "  good morning  everyone   "},
    {"post_id":13, "text": "terrible interface, crashes often"},
    {"post_id":14, "text": "Unbelievable service. 10/10"},
    {"post_id":15, "text": "meh, could be better ;("},
    {"post_id":16, "text": "http://skip.me This is a link then text"},
    {"post_id":17, "text": "WOW, this product rules!!!"},
```

```python
df.to_csv("social_media_posts.csv", index=False)
print("Created social_media_posts.csv\n")

# Step 2: clean text
df = pd.read_csv("social_media_posts.csv")
df = df.dropna(subset=["text"]).copy()
df = df.drop_duplicates(subset=["text"]).reset_index(drop=True)

def clean_text(t: str) -> str:
    t = re.sub(r"http\S+|www\S+|https\S+", "", t, flags=re.IGNORECASE)  # URLs
    t = re.sub(r"[^A-Za-z0-9\s]", " ", t)  # keep alpha-num and spaces
    t = re.sub(r"\s+", " ", t).strip()
    return t.lower()

df["clean_text"] = df["text"].astype(str).apply(clean_text)

tokenizer = nltk.word_tokenize
stop_words = set(stopwords.words("english"))
lemmatizer = WordNetLemmatizer()

def tokenize_and_lemmatize(text):
    tokens = tokenizer(text)
    tokens = [w for w in tokens if w not in stop_words and len(w) > 1]
    return [lemmatizer.lemmatize(w) for w in tokens]

df["tokens"] = df["clean_text"].apply(tokenize_and_lemmatize)

# Step 3: feature extraction
df["word_count"] = df["tokens"].apply(len)
df["sentiment"] = df["clean_text"].apply(lambda x: TextBlob(x).sentiment.polarity)
```

```
Cleaned dataset with features:

   post_id                                   text                          clean_text                                  tokens  word_count  sentiment
0        1  Love this! #sunnyday https://example.com                 love this sunnyday                       [love, sunnyday]           2   0.500000
1        2                 bad service... never again!!            bad service never again                 [bad, service, never]           3  -0.700000
2        3   OMG!!! this is the best thing EVER!! 😍        omg this is the best thing ever              [omg, best, thing, ever]           4   1.000000
3        4            Visit https://test.com now :)                          visit now                                [visit]           1   0.000000
4        6                                Just meh...                           just meh                                  [meh]           1   0.000000
5        7          JUST got a new camera, So happy.        just got a new camera so happy              [got, new, camera, happy]           4   0.468182
6        8          just got a new camera, so happy.        just got a new camera so happy              [got, new, camera, happy]           4   0.468182
7        9           Spam http://spam.org spam spam                        spam spam spam                      [spam, spam, spam]           3   0.000000
8       10                       I hate this! #fail                     i hate this fail                            [hate, fail]           2  -0.650000
9       11                       i Hate this! #fail                     i hate this fail                            [hate, fail]           2  -0.650000
10      12                 good morning  everyone                   good morning everyone                [good, morning, everyone]           3   0.700000
11      13         terrible interface, crashes often       terrible interface crashes often      [terrible, interface, crash, often]           4  -1.000000
12      14              Unbelievable service. 10/10            unbelievable service 10 10          [unbelievable, service, 10, 10]           4  -0.250000
13      15                   meh, could be better ;(                  meh could be better                     [meh, could, better]           3   0.500000
14      16   http://skip.me This is a link then text              this is a link then text                          [link, text]           2   0.000000
15      17                 WOW, this product rules!!!                 wow this product rules                 [wow, product, rule]           3   0.100000
16      18 worst experience ever, never buying again worst experience ever never buying again [worst, experience, ever, never, buying]           5  -1.000000
17      19                     happy happy happy :)                       happy happy happy                 [happy, happy, happy]           3   0.800000
18      20       Bad, bad, bad... yet okay sometimes.        bad bad bad yet okay sometimes    [bad, bad, bad, yet, okay, sometimes]           6  -0.400000

Data summary:
       word_count  sentiment
count   19.000000  19.000000
mean     3.105263  -0.005981
std      1.286457   0.607143
min      1.000000  -1.000000
25%      2.000000  -0.525000
50%      3.000000   0.000000
75%      4.000000   0.484091
max      6.000000   1.000000
```

Explanation

This task involves cleaning and preprocessing textual data from social media. Duplicate and null entries are removed to ensure data quality. Text is cleaned by removing URLs, special characters, and extra spaces using regular expressions. The cleaned text is converted to lowercase and tokenized into words. Stopwords are removed to retain meaningful words. Finally, features such as word count and sentiment score are generated for further analysis.

Conclusion

In this lab, various datasets including academic, financial, environmental, traffic, and social media data were cleaned and preprocessed using AI-assisted approaches. Techniques such as handling missing values, removing duplicates, encoding categorical variables, normalization, and feature extraction were applied effectively. These preprocessing steps significantly improve data quality and ensure better performance of machine learning models. Additionally, the use of AI-generated scripts helps in automating repetitive tasks, saving time, and reducing human errors.